Thundercomm

Empowering Every IoT Device with Our Technology



2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

Thundercomm TurboX SOM APSS GPIO TLMM v4 Pin Control

Software User Guide

Rev. V1.0
Feb 25, 2022

DN: tc-A-161410

Revision History

Revision	Date	Description
1.0	Feb 25, 2022	Initial release.

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

Copyright© 2022 Thundercomm Technology Co., Ltd. All rights reserved.

Table List

Table 2-1. TLMM_GPIO_CFGn register bit field

Table 2-2. TLMM_GPIO_LP_CFGn bit field

Table 2-3. TLMM_GPIO_IN_OUTn register bit filed

Table 2-4. TLMM_GPIO_CFGn register bit field

Table 2-5. TLMM_GPIO_INTR_STATUS register bit field

Table 2-6. TLMM_GPIO_ID_STATUS register bit field

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

Copyright© 2022 Thundercomm Technology Co., Ltd. All rights reserved.

About This Document

- This document is applicable to the following products:
  - TurboX C4290/CM4290 SOM
  - Turbox C2290/CM2290 SOM
  - TurboX S605 SBC
  - TurboX C410/C610 SOM
  - TurboX D660/D660 Pro SOM
- Illustrations in this documentation might look different from your product.
- Depending on the model, some optional accessories, features, and software programs might not be available on your device.
- Depending on the version of operating systems and programs, some user interface instructions might not be applicable to your device.
- Documentation content is subject to change without notice. Thundercomm makes constant improvements on the documentation of the products, including this guidebook.
- Function declarations, function names, type declarations, attributes, and code samples appear in a different format, for example, `cp armcc armcpp`.
- Code variables appear in angle brackets, for example, `<number>`.
- Button, tool, and key names appear in bold font, for example, click **Save** or press **Enter**.
- Commands to be entered appear in a different font; on the host computer use $ as shell prompt, while on the target device use # as shell prompt, for example,

```
$ adb devices
# logcat
```

- Part of the code that does not contain instructions appear in a different format, for example,

```
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0777", GROUP="adm"
```

- Folders and files are formatted in italic, for example, *turbox_flash_flat.sh*.

Thundercomm Confidential
2025-01-31 9:16:22 AM CST
Stefan_ucam.mo@fusion.com

Copyright© 2022 Thundercomm Technology Co., Ltd. All rights reserved.

Table of Contents

Chapter 1. Overview ..... - 2 -

Chapter 2. GPIO hardware overview ..... - 3 -

2.1. GPIO pin configuration and pin function ..... - 3 -

2.2. GPIO pin state ..... - 3 -

2.3. GPIO pad structure ..... - 4 -

2.4. GPIO configuration register ..... - 5 -

2.5. GPIO input and output register ..... - 6 -

2.6. GPIO interrupt configuration register ..... - 7 -

2.7. GPIO interrupt status register ..... - 8 -

2.8. GPIO ID status register ..... - 8 -

Chapter 3. GPIO TLMM v4 programming ..... - 9 -

3.1. GPIO programming software model prior to TLMM v4 ..... - 9 -

3.2. Software model for GPIO programming TLMM v4 ..... - 9 -

Chapter 4. GPIO pin control software model ..... - 10 -

4.1. GPIO pinctrl config ..... - 10 -

4.2. GPIO as a common input and output port ..... - 11 -

Chapter 5. GPIO pin control software use cases ..... - 12 -

5.1. Pins driven by controllers ..... - 12 -

5.2. Pins driven by software (bit bang) ..... - 17 -

5.3. Pins used as interrupt sources ..... - 19 -

5.4. Pins used as clock sources ..... - 20 -

Chapter 6. Debug via adb Commands ..... - 22 -

6.1. Map IO port GPIO to file system ..... - 22 -

6.2. Check pinctrl status ..... - 22 -

Appendix 1. Notices ..... - 23 -

Appendix 2. Trademarks ..... - 25 -

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

Chapter 1. Overview

The Linux pin control framework enables you to configure the GPIOs (General-purpose Inputs/Outputs) in the APSS (Application Processor Subsystem). The framework is designed specifically for embedded applications on SoC (System-on-Chip) where the outfacing pins are considered precious and limited resources.

A PinMuxing (Pin Multiplexing) logic allows a pin or a group of pins to be reused for different purposes. Each use case of these pins requires different attributes to be configured to drive the signal on the pins. The pin control and PinMux framework allow users to specify different configuration attributes and define multiple uses or functions for a group of pins.

This document explains the pin control framework to configure the SoC pin resources on the device. It describes different use cases of the SoC pins for this chipset.

The document is intended for Linux device driver developers and software engineers who want to configure GPIOs in the APSS.

➲ **NOTE**: Basic knowledge of the framework and the underlying driver implementation.

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

Chapter 2. GPIO hardware overview

2.1. GPIO pin configuration and pin function

GPIO pin configuration programming is an attribute that governs the nature of a signal to be driven on the lines. Typical configurations include drive strength, pull values, direction, and so on. GPIO pin function can be defined as the use of the signal to be driven on the GPIO pins.

2.2. GPIO pin state

The framework implementation employs the concept of GPIO pin states. A state comprises configurations and functions to be applied on a group of pins. An owner of one or more pins must furnish the information that comprises the state of a pin group. The framework allows competing owners, that is, independent clients that want to use the same GPIO pins for different purposes can furnish the respective GPIO pin state information.

There is no limit to the number of states an owner can define for a GPIO pin. However, owners typically define the following three states:

- Default
- Active
- Sleep

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

2.3. GPIO pad structure

A GPIO pin can be configured as general input and output, or one of the several alternate functions. When a GPIO pin is used as an input it can also be set as an interrupt source.

A subset of GPIOs can generate interrupts that can bring the mobile device out of deep sleep (XO shutdown or VDD minimization). GPIOs can also be configured to have internal pull circuits (up, down, or no-pull) and different driving strengths.

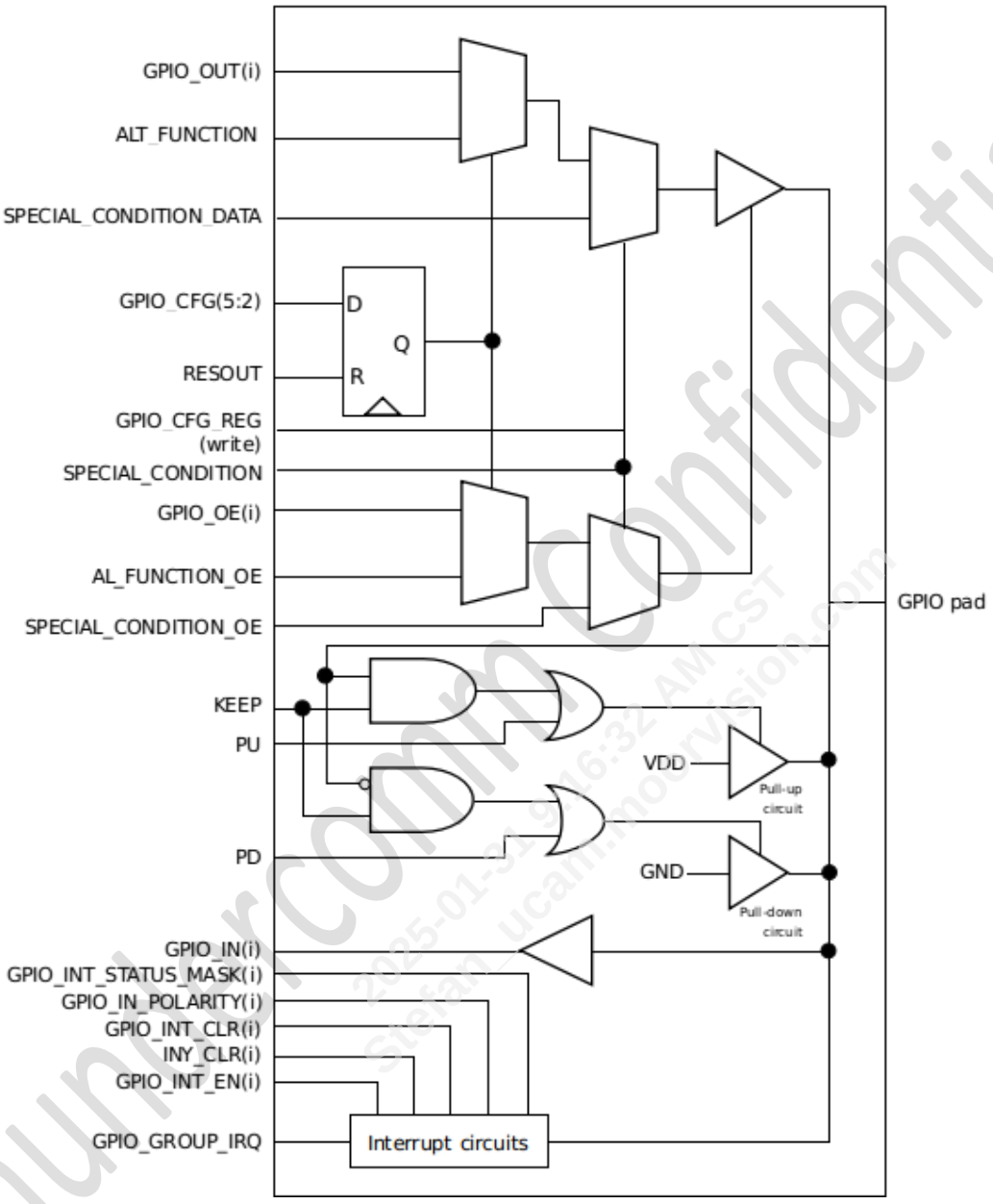


Figure 2-1.GPIO Pad Structure

Thundercomm Confidential

2.4. GPIO configuration register

The TLMM_GPIO_CFGn register controls the configuration of GPIOs. Each register controls the configuration of one GPIO.

Table 2-1. TLMM_GPIO_CFGn register bit field

Bit	Name	Description
31:13	Reserved	Reserved field
12	EGPIO_ENABLE	Controls the EGPIO feature: • 0: EGPIO Disabled • 1: EGPIO Enabled
11	EGPIO_PRESENT	Reads if the EGPIO feature is enabled or not
10	GPIO_HIHYS_EN	Controls the hihys_en for GPIO[n]: • 0: Disable • 1: Enable
9	GPIO_OE	Controls the OE (Output Enable) for GPIO[n] when GPIO[n] is in GPIO mode: • 0: Input • 1: Output
8:6	DRV_STRENGTH	Controls the GPIO drive strength. This applies regardless of the FUNC_SEL field selection. Increase of the driving strength can enhance the driving capability and increase the power consumption. • 000: 2mA • 001: 4mA • 010: 6mA • 011: 8mA • 100: 10mA • 101: 12mA • 110: 14mA • 111: 16mA
5:2	FUNC_SEL	Many of the GPIOs have one or more functional hardware interfaces behind them. This field controls how it is used. Set this to the appropriate value for the function desired.
1:0	GPIO_PULL	The GPIO can be configured to employ an internal weak pull up, pull down, or keeper function. This applies regardless of the FUNC_SEL field selection. This is a default state for GPIO. Output is usually set to no_pull. GPIO is input, default is high and set to pull_up, otherwise set to pull_down. • 00: NO_PULL (Disables all pull) • 01: PULL_DOWN (Weak Pull-Down) • 10: KEEPER (Weak Keeper) • 11: PULL_UP (Weak Pull-Up)

The TLMM_GPIO_LP_CFGn registers are used to store low-power configuration of GPIOs. These registers are initialized during bootup. When the driver invokes TLMM (Top-level Mode Multiplexer) API to place a GPIO into inactive state, the configuration data stored in this register is retrieved and written to the TLMM_GPIO_CFGn register.

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
qiyan_ucam.moorvision.com

Table 2-2. TLMM_GPIO_LP_CFGn bit field

Bit	Name	Description
13	GPIO_VALID	0: NO 1: Yes
12	EGPIO_ENABLE	-
11	EGPIO_PRESENT	-
10	GPIO_OUT	0: Low 1: High
9	GPIO_OE	-
8:6	DRV_STRENGTH	• 000: 2mA • 001: 4mA • 010: 6mA • 011: 8mA • 100: 10mA • 101: 12mA • 110: 14mA • 111: 16mA
5:2	FUNC_SEL	-
1:0	GPIO_PULL	• 00: NO_PULL • 01: PULL_DOWN • 10: KEEPER • 11: PULL_UP

2.5. GPIO input and output register

When the FUNC_SEL bit of a GPIO is set to 0, the GPIO is used as a general-purpose pin. The GPIO_OE bit of the TLMM_GPIO_CFGn register controls the value of the bit.

- If set to 1, the pin is used as an output pin and bit 1 of the TLMM_GPIO_IN_OUTn register controls its state.
- If set to 0, the pin is used as an input pin. The state of a GPIO can be read from bit 0 in the same register.

Table 2-3. TLMM_GPIO_IN_OUTn register bit filed

Bit	Name	Description
1	GPIO_out	Controls the output state of an output pin • 0: Low • 1: High
0	GPIO_IN	Reads the state of an input pin

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
sean_ucam.moorvision.com

2.6. GPIO interrupt configuration register

All GPIOs can be set as interrupt sources. The TLMM_GPIO_INTR_CFGn register controls the GPIO interrupt functionality.

Table 2-4. TLMM_GPIO_CFGn register bit field

Bit	Name	Description
31:9	Reserved	Reserved field
8	DIR_CONN_EN	Notifies the TLMM that GPIO[n] is being used as a direct connect interrupt: • 0: Disables the GPIO as a direct connect interrupt • 1: Enables the GPIO as a direct connect interrupt
7:5	TAEGET_PROC	Determines the processor to which a summary interrupt from GPIO[n] gets routed: • 000: Sensor, routing the GPIO[n] signal to the sensor's summary interrupt • 001: LPA_DSP, routing the GPIO[n] signal to the LPASS summary interrupt • 010: RPM, routing the GPIO[n] signal to the RPM summary interrupt • 011: APSS, routing the GPIO[n] signal to the APSS summary interrupt • 100: MSS, routing the GPIO[n] signal to the MSS summary interrupt • 101: TrustZone, routing the GPIO[n] signal to the TrustZone summary interrupt • 110: cDSP, routing the GPIO[n] signal to the cDSP summary interrupt • 111: None, indicating it does not route to any processor subsystem; this is the default setting.
4	INTR_RAW_STATUS_EN	Enables the RAW status for the summary interrupt on this GPIO; this is a power-saving mechanism and should be left disabled unless necessary: • 1: Enable • 0: Disable
3:2	INTR_DECT_CTL	Controls the edge or level detection of the interrupt controller: 00: LEVEL, level sensitive 01: POS_EDGE, positive edge sensitive 10: NEG_EDGE, negative edge sensitive 11: BUAL_EDGE, sensitive to both edges
1	INTR_POL_CTL	Controls the polarity detection of the interrupt controller: Polarity 1 corresponds to active high while Polarity 0 corresponds to active low. • 0: POLARITY_0 • 1: POLATRITY_1
0	INTR_ENABLE	Controls the generation of summary interrupt: • 1: Enable • 0: Disable

The trigger type of the interrupt is controlled by INTR_POL_CTL and INTR_DETC_CTL. Setting or clearing the INTR_ENABLE bit field enables or disables the corresponding GPIO interrupt.

Thundercomm Confidential

2.7. GPIO interrupt status register

The interrupt status for GPIO[n] is recorded in the TLMM_GPIO_INTR_STATUS.

Table 2-5. TLMM_GPIO_INTR_STATUS register bit field

Bit	Name	Description
31:1	Reserved	Reserved
0	INTR_STATUS	• When read, returns the interrupt status of GPIO[n] ▫ 0: The interrupt is not active ▫ 1: The interrupt is active • When written, clears or sets the interrupt for GPIO[n] ▫ 0: Clear the interrupt ▫ 1: Set the interrupt

2.8. GPIO ID status register

The ID status register identifies the tile in which GPIO ID is present.

The GPIO ID status for GPIO[n] is recorded in the TLMM_GPIO_ID_STATUS.

When read, this register returns the ID status of the corresponding GPIO.

The GPIO belongs to particular tile when the register returns 1; when the register returns 0, GPIO does not belong to that tile.

Table 2-6. TLMM_GPIO_ID_STATUS register bit field

Bit	Name	Description
31:1	Reserved	Reserved
0	GPIO_ID_STATUS	Read the state of an ID pin

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_liu@mobilevision.com

Chapter 3. GPIO TLMM v4 programming

SoCs have a PinMux controller known as TLMM. The current version of TLMM is v4.

3.1. GPIO programming software model prior to TLMM v4

Prior to TLMM v4, the GPIO library was used to configure pins and apply specific functions to them.

- The pin configurations were defined in the *board -<soc>- gpiomux.c* file. These configurations were installed during kernel initialization.
- Clients defined at least two sets of configurations: **Active** and **Sleep**.
- The *gpio_request()* and *gpio_free()* APIs were used to install and revert the **Active** and **Sleep** pin configuration states at runtime.

In few cases, a separate API was provided to the clients to program more than two configurations.

3.2. Software model for GPIO programming TLMM v4

TLMM v4 supports multiplexing and configuration of different types of pins. Any SoC that incorporates TLMM v4 controller can only have pins of types supported by TLMM v4. These types have different register semantics, they also have their own protocol-specific pin attributes in certain cases (attributes different from drive strength, pull, direction, and so on). In the context of pin control and based on the use cases (pins driven by controllers, software, clocks, and interrupts), this document applies to the GPIO supported by TLMM v4.

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

Chapter 4. GPIO pin control software model

4.1. GPIO pinctrol config

A pin control-based software model differs in the following ways:

- All pin configurations and TLMM information are defined in a SOC-specific file, *<soc>-pinctrl.dtsi*. This file contains the pin groupings as well as the configurations to be applied to them.
- Individual client device tree nodes reference the configuration nodes defined in *<soc>-pinctrl.dtsi*. It includes the state of the pins. A client can have any number of states and any number of configuration nodes in a state.
- The state information of a GPIO is parsed with a call to `devm_pinctrl_get()`.
- A client can then choose to install a given state using `pinctrl_lookup_state()` and `pinctrl_select_state()`.

In the following example, *devicetree-4.19/qcom/bengal-pinctrl.dtsi* is a sample TLMM PinMux node containing the pin types:

```
C
&soc {
    tlmm: pinctrl@400000 {
        compatible = "qcom,bengal-pinctrl";
        reg = <0x400000 0xc00000>;
        interrupts = <GIC_SPI 227 IRQ_TYPE_LEVEL_HIGH>;
        gpio-controller;
        #gpio-cells = <2>;
        interrupt-controller;
        #interrupt-cells = <2>;

        qupv3_se4_2uart_pins: qupv3_se4_2uart_pins {
            qupv3_se4_2uart_active: qupv3_se4_2uart_active {
                mux {
                    pins = "gpio12", "gpio13";
                    function = "qup4";
                };

                config {
                    pins = "gpio12", "gpio13";
                    drive-strength = <2>;
                    bias-disable;
                };
            };

            qupv3_se4_2uart_sleep: qupv3_se4_2uart_sleep {
                mux {
                    pins = "gpio12", "gpio13";
                    function = "gpio";
                };

                config {
                    pins = "gpio12", "gpio13";
                    drive-strength = <2>;
                    bias-pull-down;
                };
            };
        };
    };
};
```

Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

4.2. GPIO as a common input and output port

Pins is used as normal GPIO, as input and output pins.

- gpio_request: request GPIO.
- gpio_direction_output: config GPIO as output.
- gpio_direction_input: config GPIO as input.
- gpio_set_value: control output high level or low level.

```
Soc {
    ...
    client-reset-gpio= <&tlmm 134 0x0>;
    client-ps-gpio = <&tlmm 136 0x0>;
    ...
};

client_driver_probe(struct platform_device *pdev) {
   ...
   client_reset_gpio = of_get_named_gpio(node, "client-reset-gpio", 0);
   if (!gpio_is_valid(client_reset_gpio)) {
      pr_err("reset gpio not specified\n");
    ret = -EINVAL;
   }
   ret = gpio_request(client_reset_gpio, "reset-gpio");
   if (ret) {
      pr_err("reset gpio request failed\n");
   }
   ret = gpio_direction_output(client_reset_gpio, 0);
   if (ret) {
      pr_err("reset gpio direction failed\n");
   }

   client_ps_gpio = of_get_named_gpio(node, "client-ps-gpio", 0);
   if (!gpio_is_valid(client_ps_gpio ))
    pr_debug("ps gpio not specified\n");
   else
    pr_debug("client_ps_gpio =%d\n", client_ps_gpio );

   ret = gpio_request(client_ps_gpio, "ps-gpio");
   if (ret) {
      pr_err("ps gpio request failed\n");
   }
   ret = gpio_direction_input(client_ps_gpio);
   if (ret) {
      pr_err("ps gpio direction failed\n");
   }
   ...
   gpio_set_value(client_reset_gpio, 1);
   msleep(20);
   gpio_set_value(client_reset_gpio, 0);
   msleep(20);
   gpio_set_value(client_reset_gpio, 1);
}
```

2025/10/31 9:16:32 AM CST stefan_ucam.moorvision.com

Chapter 5. GPIO pin control software use cases

The usage of pins on SoCs can be divided into the following types:

- Pins driven by controllers, for example, universal asynchronous receiver/transmitter (UART), serial peripheral interface (SPI), and inter-integrated circuit (I^2C).
- Pins used by software for bit banging; typically, the devices attached externally to the board, for example, display panels.
- Pins that are used as a source of interrupts; typically attached externally to the board, for example, Ethernet.
- Pins as clock sources.

In a few instances, OEMs use the same group of pins for use cases 1 and 2. In such instances, these two cases are modeled as extra states in the device tree.

5.1. Pins driven by controllers

This case continues the usage of the example provided in GPIO pin control software model.

5.1.1. Configure GPIO pins during bootup only

Step 1.Add a pin grouping node to the *bengal-pinctrl.dtsi* file.

```
&soc {
    tlmm: pinctrl@400000 {
        ...
        Client1_pins {
            Client1_default: client1_default {
                mux {
                    pins = "gpio54";
                    function = "gpio";
                };

                config {
                    pins = "gpio54";
                    drive-strength = <2>; /* 2mA */
                    bias-pull-down;
                    input-enable;
                };
            };
        };
    };
};
```

Step 2.Modify the client node in the `device tree()`.

```
&soc {
    ...
    Client1 {
        ...
        /*Note the default state is programmed by the kernel at the time of
        kernel bootup. No driver changes are necessary since at probe time the
        default state would already be programmed in TLMM*/
        pinctrl-states = "default";
        /* Use phandle reference to default configuration node */
        pinctrl-0 = <&Client1_default>;
    };
};
```

Confidential 2025-01-31 9:16:32 AM CST stefan_ucam.moonvision.com

5.1.2. Configure GPIO pins for Active and Sleep states (runtime)

In this case, a controller driver has **Active** state for pins during transactions or as the default state, and Sleep state when suspending the system. The driver must install the correct state accordingly.

To configure general-purpose pins for Active and Sleep states:

Step 1. Define the pin group node containing **Active** and **Sleep** states.

```
&soc {
    tlmm: pinctrl@400000 {
        ...
        qupv3_se0_i2c_pins: qupv3_se0_i2c_pins {
            qupv3_se0_i2c_active: qupv3_se0_i2c_active {
                mux {
                    pins = "gpio0", "gpio1";
                    function = "qup0";
                };

                config {
                    pins = "gpio0", "gpio1";
                    drive-strength = <2>;
                    bias-disable;
                };
            };

            qupv3_se0_i2c_sleep: qupv3_se0_i2c_sleep {
                mux {
                    pins = "gpio0", "gpio1";
                    function = "gpio";
                };

                config {
                    pins = "gpio0", "gpio1";
                    drive-strength = <2>;
                    bias-pull-up;
                };
            };
        };
    };
};
```

Step 2. Modify the client node to include pinctrl states. Delete any explicit references to the GPIOs using the GPIO chip node.

```
SOC {
    ...
    Client2 {
        ...
        pinctrl-names = "default", "sleep";
        pinctrl-0 = <&qupv3_se0_i2c_active>;
        pinctrl-1 = <&qupv3_se0_i2c_sleep>;
    };
};
```

Step 3. Modify the driver to pinctrl interface.

```
static int client2_probe(struct platform_device *pdev) {
    struct pinctrl_state *set_state;
    struct device *dev = &pdev->dev;
    int ret = 0;
    ...
    /* Try to obtain pinctrl handle */
    dev->pins->p = devm_pinctrl_get(dev);
    /* Lookup the active configuration */
    set_state = pinctrl_lookup_state(dev->pins->p, "default");
    if(IS_ERR(set_state)) {
        ret = PTR_ERR(set_state);
        dev_err(dev, "Can't get default pinctrl state, ret %d\n", ret);
    } else
        ret = pinctrl_select_state(dev->pins->p, set_state);
```

2025-01-31 19:16:32 AM CST
qianfan_ucan moorvision.com

```
}

/* Install sleep configuration in runtime suspend function*/
static int client2_suspend(struct device *dev) {
    struct pinctrl_state *set_state;
    set_state = pinctrl_lookup_state(dev->pins->p, "sleep");
    if(IS_ERR(set_state)) {
        ret = PTR_ERR(set_state);
        dev_err(dev, "Can't get sleep pinctrl state, ret %d\n", ret);
    } else
        ret = pinctrl_select_state(dev->pins->p, set_state);
}
```

5.1.3. Configure GPIO pin groups with different function settings (runtime)

The function applied to a pin group changes as part of installing a new state. In these cases, the pin group is modeled as two separate pin groups using the same pins; the driver chooses to install a new state for the pin groups with different or same configurations, however, with different pin functions.

The following example shows different pin functions applied to the same pin group:

```
&soc {
    tlmm: pinctrl@400000 {
        ...
        qupv3_se3_4uart_pins: qupv3_se3_4uart_pins {
            qupv3_se3_default_ctsrtsrx:
                qupv3_se3_default_ctsrtsrx {
                mux {
                    pins = "gpio8", "gpio9", "gpio11";
                    function = "gpio";
                };

                config {
                    pins = "gpio8", "gpio9", "gpio11";
                    drive-strength = <2>;
                    bias-pull-down;
                };
            };

            qupv3_se3_default_tx:
                qupv3_se3_default_tx {
                mux {
                    pins = "gpio10";
                    function = "gpio";
                };

                config {
                    pins = "gpio10";
                    drive-strength = <2>;
                    bias-pull-up;
                };
            };

            qupv3_se3_ctsrx: qupv3_se3_ctsrx {
                mux {
                    pins = "gpio8", "gpio11";
                    function = "qup3";
                };

                config {
                    pins = "gpio8", "gpio11";
                    drive-strength = <2>;
                    bias-disable;
                };
            };

            qupv3_se3_rts: qupv3_se3_rts {
                mux {
                    pins = "gpio9";
```

2025-01-31 9:16:32 AM CST
stefan_ucam.moorvision.com

```
                function = "qup3";
            };

            config {
                pins = "gpio9";
                drive-strength = <2>;
                bias-pull-down;
            };
        };

        qupv3_se3_tx: qupv3_se3_tx {
            mux {
                pins = "gpio10";
                function = "qup3";
            };

            config {
                pins = "gpio10";
                drive-strength = <2>;
                bias-pull-up;
            };
        };
    };
};
```

5.1.4. Configure GPIO pin groups for SDC pins (runtime)

TLMM v4 supports dedicated SDC pin configurations for SDC1 and SDC2. These pins are used for clock, command, and data signals as part of the SDC specification.

Model it as six SDC type pins:

- Pins 0 to 2 correspond to clock, command, and data of SDC1.
- Pins 3 to 5 correspond to clock, command, and data of SDC2.

The following example describes the configuration of SDC pin types.

Step 1.Extend the *PinMux* node to define pins for SDC1 and SDC2.

```
&soc {
    tlmm: pinctrl@400000 {
        ...
        /* SDC pin type */
        sdc1_clk_on: sdc1_clk_on {
            config {
                pins = "sdc1_clk";
                bias-disable;       /* NO pull */
                drive-strength = <16>;  /* 16 MA */
            };
        };

        sdc1_clk_off: sdc1_clk_off {
            config {
                pins = "sdc1_clk";
                bias-disable;       /* NO pull */
                drive-strength = <2>;   /* 2 MA */
            };
        };

        sdc1_cmd_on: sdc1_cmd_on {
            config {
                pins = "sdc1_cmd";
                bias-pull-up;       /* pull up */
                drive-strength = <10>;  /* 10 MA */
            };
        };

        sdc1_cmd_off: sdc1_cmd_off {
            config {
```

```
            pins = "sdc1_cmd";
            bias-pull-up;      /* pull up */
            drive-strength = <2>;   /* 2 MA */
        };
    };

    sdc1_data_on: sdc1_data_on {
        config {
            pins = "sdc1_data";
            bias-pull-up;      /* pull up */
            drive-strength = <10>;  /* 10 MA */
        };
    };

    sdc1_data_off: sdc1_data_off {
        config {
            pins = "sdc1_data";
            bias-pull-up;      /* pull up */
            drive-strength = <2>;   /* 2 MA */
        };
    };

    sdc1_rclk_on: sdc1_rclk_on {
        config {
            pins = "sdc1_rclk";
            bias-pull-down; /* pull down */
        };
    };

    sdc1_rclk_off: sdc1_rclk_off {
        config {
            pins = "sdc1_rclk";
            bias-pull-down; /* pull down */
        };
    };

    sdc2_clk_on: sdc2_clk_on {
        config {
            pins = "sdc2_clk";
            bias-disable;      /* NO pull */
            drive-strength = <16>;  /* 16 MA */
        };
    };

    sdc2_clk_off: sdc2_clk_off {
        config {
            pins = "sdc2_clk";
            bias-disable;      /* NO pull */
            drive-strength = <2>;   /* 2 MA */
        };
    };

    sdc2_cmd_on: sdc2_cmd_on {
        config {
            pins = "sdc2_cmd";
            bias-pull-up;      /* pull up */
            drive-strength = <10>;  /* 10 MA */
        };
    };

    sdc2_cmd_off: sdc2_cmd_off {
        config {
            pins = "sdc2_cmd";
            bias-pull-up;      /* pull up */
            drive-strength = <2>;   /* 2 MA */
        };
    };

    sdc2_data_on: sdc2_data_on {
        config {
            pins = "sdc2_data";
```

2025-01-31 9:16:32 AM CST Stephen_ucam.moorvision.com

```
            bias-pull-up;      /* pull up */
            drive-strength = <10>;  /* 10 MA */
        };
    };

    sdc2_data_off: sdc2_data_off {
        config {
            pins = "sdc2_data";
            bias-pull-up;      /* pull up */
            drive-strength = <2>;   /* 2 MA */
        };
    };
};
};
```

Step 2. Update SDHC (Secure Digital High Capacity) nodes to point to the pin configurations defined in Step 1.

Delete the old configurations and add your configuration to *bengal-idp.dtsi*.

```
&sdhc_1 {
    ...
    pinctrl-names = "active", "sleep";
    pinctrl-0 = <&sdc1_clk_on &sdc1_cmd_on &sdc1_data_on &sdc1_rclk_on>;
    pinctrl-1 = <&sdc1_clk_off &sdc1_cmd_off &sdc1_data_off &sdc1_rclk_off>;
};
```

Step 3. Update the driver to parse device tree for pinctrl nodes as shown in 5.1.2. Configure GPIO pins for Active and Sleep states (runtime).

5.2. Pins driven by software (bit bang)

Two frameworks are applicable to this use case:

- *Pinctrl* to configure the pin
- *GPIO lib* to drive the required signal values on the pin

The general-purpose pin type on the TLMM node (currently the only pin type on TLMM v4 to support GPIO functionality) must be augmented with a GPIO chip controller node. Extend the TLMM PinMux node from *GPIO pin control software model*, as shown in the following example. The GPIO controller node is used as a handle to specify GPIO pins in client nodes. Such pins are typically used outside the SoC, for example, display panel, Ethernet. These pin group nodes are added through extension of the *tlmm_pinmux* node in the corresponding *<soc>-board.dtsi* file.

Configuration of GPIO pins for software bit banging (runtime) shows configuration of the *TLMM PinMux* node with the GPIO controller for the GPIO pin type.

Example: GPIO client and pin configuration for Active and Sleep states

Step 1. Extend the *tlmm_pinmux* node in the corresponding *<soc>-<board>.dtsi* file to add pin configuration node for **Active** and **Sleep** states.

```
&soc {
    tlmm: pinctrl@400000 {
        ...
        Client3_pins {
            /* pin group node */
            /* uses general purpose pins */
            client3_pin_active: client3_pin_active {
                mux {
                    pins = "gpio64";
                    function = "gpio";
                };

                config {
                    pins = "gpio64";
                    drive-strength = <16>;
                    bias-pull-down;
                    output-high;
```

```
            };
        };

        client3_pin_sleep: client3_pin_sleep {
            mux {
                pins = "gpio64";
                function = "gpio";
            };

            config {
                pins = "gpio64";
                drive-strength = <16>;
                bias-disable;
                output-low;
            };
        };
    };
};
```

Step 2. Modify the client node to include pinctrl states. Also, modify any explicit references to the GPIO chip handler from Pins driven by software (bit bang).

```
Soc {
    ...
    Client3 {
        ...
        pinctrl-names = "active", "sleep";
        /* Use phandle referrence to active and sleep configurations to
        define the active and sleep pin states */
        pinctrl-0 = <&client3_pin_active>;
        pinctrl-1 = <&client3_pin_sleep>;
        /* Use phandle of correct gpio controller */
        client3-gpio-1 = <&tlmm 64 0>;
    };
};
```

Step 3. Modify the driver to search for pinctrl states only. The GPIOs are probed even after the pinctrl states are found.

```
int client3_probe(struct platform_device *pdev)
{
    struct pinctrl_state *set_state;
    struct device *dev = &pdev->dev;

    ...
    /* Try to obtain pinctrl handle */
    dev->pins->p = devm_pinctrl_get(dev);
    /* Lookup the active configuration */
    set_state = pinctrl_lookup_state(dev->pins->p, "active");
    if(IS_ERR(set_state))
        goto fail;
    client3_data->active_state = set_state;
    set_state = pinctrl_lookup_state(dev->pins->p, "sleep");
    client3_data->sleep_state = set_state;

    /* Modify the driver to look for pinctrl states only.
    The GPIOs are probed even after the pinctrl statesare found. */
    ret = pinctrl_select_state(dev->pins->p, client3_data->actice_state);
    ...

    /* gpio_probe: */
    client3_data->reset_gpio = of_get_named_gpio(node, “client3_gpio_1”);
    gpio_request(client3_data->reset_gpio, “reset”);
    ...
}
```

Step 4. Install the required configuration before bit banging.

```
int client3_driver_signal(bool active)
{
    ...
```

```
    if (active)
        ret = pinctrl_select_state(dev->pins->p, client3_data->actice_state);
    else
        ret = pinctrl_select_state(dev->pins->p, client3_data->sleep_state);
    gpio_direction_out(client3_data->reset_gpio);
    gpio_set_value(client3_data->reset_gpio, 1);
}
```

5.3. Pins used as interrupt sources

5.3.1. Overview

The general-purpose pins present on a TLMM block can be used as interrupt sources. The *PinMux* node must be augmented with interrupt controller attributes to support this use case. Since this is supported by general-purpose pin types, augment the corresponding GPIO controller node with interrupt controller attributes.

5.3.2. Configure GPIO pins as interrupt sources during bootup only shows the PinMux node with interruptcontroller attributes.

5.3.2. Configure GPIO pins as interrupt sources during bootup only

When pins are used as interrupt sources, the pin attributes are configured once, at bootup. The pin is then used as an interrupt source. To accomplish this, the pin configuration must be modeled as a default state of the pin. Also, since these devices are present as attachments to the SoC on a board, their pin groups are added through extension of the *PinMux* node in the corresponding board-specific device tree source file.

The following example shows an interrupt request (IRQ) client.

Step 1.Extend the *tlmm_pinmux* node in the *<soc>-<board>.dtsi* file to add the pin configuration.

```
&soc {
    tlmm: pinctrl@400000 {
        ...
        client4_pins {
            client4_default: client4_default {
                mux {
                    pins = "gpio54";
                    function = "gpio";
                };

                config {
                    pins = "gpio54";
                    drive-strength = <2>; /* 2mA */
                    bias-pull-down; /* pull down */
                    input-enable;
                };
            };
        };
    };
};
```

Step 2.Update the client device node to point to the configuration node.

```
Soc {
    ...
    client4 {
        pinctrl-states = "default";
        pinctrl-0 = <&client4_default>;
        interrupt-parent = <&tlmm>;
        /*Add reference to the interrupt controller node defined in example above*/
```

```
        interrupts = <54 0>;
        interrupt-names = "lt_irq";
        lt,irq-gpio = <&tlmm 54 0x0>;
    };
};
```

Step 3. Driver changes to apply pin configuration is not needed as default pin configuration is installed by the kernel at bootup.

```
/* probe */
client4_data->gpio_irq = of_get_named_gpio(np, "lt,irq-gpio", 0);
gpio_request(client4_data->gpio_irq, "client4-irq-gpio");

client4_data->irq = gpio_to_irq(pdata->gpio_irq);
ret = request_threaded_irq(client4_data->irq, NULL, client4_irq_thread_handler,
            IRQF_TRIGGER_FALLING | IRQF_ONESHOT, "client4", client4_data);
```

5.4. Pins used as clock sources

5.4.1. Overview

Few general-purpose pins present on a TLMM block can be used as clock sources. The *PinMux* node must be augmented with clock controller attributes to support this use case. Since this is supported by GPIO pin types, augment the corresponding GPIO controller node with clock controller attributes.

5.4.2. Configure GPIO pins as clock sources shows the PinMux node with clock controller attributes.

5.4.2. Configure GPIO pins as clock sources

When pins are used as clock sources, the pin attributes are configured once at bootup or configured dynamically. If the pin needs to be configured at bootup only, then the pin configuration must be modeled as a default state of the pin, otherwise it can define active and sleep configurations.

Also, since these devices are present as attachments to the SoC on a board, their pin groups are added through extension of the PinMux node in the corresponding board-specific device tree source file.

The following example shows a clock client:

Step 1. Extend the *tlmm_pinmux* node in the *<soc>-<board>.dtsi* file to add the pin configuration.

```
&soc {
    tlmm: pinctrl@400000 {
        ...
        client5_pins {
            client5_active: client5_active {
                mux {
                    pins = "gpio20";
                    function = "cam_mclk";
                };

                config {
                    pins = "gpio20";
                    drive-strength = <2>; /* 2 MA */
                    bias-disable; /* No PULL */
                };
            };

            client5_suspend: client5_suspend {
                mux {
                    pins = "gpio20";
                    function = "cam_mclk";
                };

                config {
                    pins = "gpio20";
                    bias-pull-down; /* PULL DOWN */
```

```
                drive-strength = <2>; /* 2 MA */
            };
        };
    };
};
```

Step 2.Update the client device node to point to the pin configuration node.

```
Soc {
    ...
    client5 {
        ...
        pinctrl-states = "active", "suspend";
        pinctrl-0 = <&client5_active>;
        pinctrl-1 = <&client5_suspend>;
        clocks = <&gcc gp1_clk_src>;
        clock-names = "gp1_clk";
    };
};
```

```
static const struct freq_tbl ftbl_gp1_clk_src[] = {
    F(19200000, P_BI_TCXO, 1, 0, 0),
    F(24000000, P_GPLL9_OUT_MAIN, 1, 1, 24),
    F(64000000, P_GPLL9_OUT_MAIN, 1, 1, 9),
    { }
};

//You need use this test code changes in probe function
int client5_probe(struct platform_device *pdev)
{
    struct pinctrl_state *set_state;
    struct device *dev = &pdev->dev;
    struct clk *pwm_clk = NULL;
    int ret;
    ...
    /* Try to obtain pinctrl handle */
    dev->pins->p = devm_pinctrl_get(dev);
    set_state = pinctrl_lookup_state(dev->pins->p, "active");
    if(IS_ERR(set_state))
        goto fail;
    client5_data->active_state = set_state;
    set_state = pinctrl_lookup_state(dev->pins->p, "suspend");
    client5_data->sleep_state = set_state;

    ret = pinctrl_select_state(dev->pins->p, client5_data->actice_state);
    pwm_clk = clk_get(cpu_dev, "gp1-clk");
    if (IS_ERR(pwm_clk)) {
        dev_err(dev, "%s: Get pwm_clk error\n", __func__);
        return -ENODEV;
    }

    //change the clock frequency by using the below piece of code
    ret = clk_set_rate(pwm_clk, (unsigned long)10000);
    if (ret < 0) {
        dev_err(cpu_dev, "%s: can't set pwm_clk rate 10KHz (%d)\n", __func__, ret);
        clk_put(pwm_clk);
        return ret;
    }

    ret = clk_prepare_enable(pwm_clk);
    if (ret < 0) {
        dev_err(cpu_dev, "%s: Can't enable pwm_clk %d\n", __func__, ret);
        clk_put(pwm_clk);
        return ret;
    }
}
```

Chapter 6. Debug via adb Commands

6.1. Map IO port GPIO to file system

You can map IO port GPIO to file system through the /sys/ file interface. The directory that controls the GPIO is located at */sys/class/gpio*. To apply this method, make sure that the GPIO is not requested in the kernel.

- “export”: Triggers an export control of a GPIO from the kernel to userspace by writing its number to this file.
- "direction": Reads as either "in" or "out". This value may normally be written. Writing as "out" defaults to initializing the value as low.
- "value": Reads as either 0 (low) or 1 (high). If the GPIO is configured as an output, this value may be written; any non-zero value is treated as high.

Example: GPIO10

```
/sys/class/gpio/gpiochip911# cat label       /* Confirm offset */
400000.pinctrl
/sys/class/gpio# echo 921 > export    /* export */
/sys/class/gpio/gpio921# echo out > direction     /* set direction */
/sys/class/gpio/gpio921# cat direction      /* check direction */
/sys/class/gpio/gpio921# echo 1 > value    /* output high */
/sys/class/gpio/gpio921# cat value    /* check output value */
/sys/class/gpio# echo 921 > unexport    /* unexport */
```

6.2. Check pinctrl status

You can check the status of pinctrl in */sys/kernel/debug/pinctrl/400000.pinctrl*.

- *cat pingroups*: get pin group information
- *cat pinmux-pins*: get pin usage
- *cat pinmux-functions*: groups that support this function
- *cat pins*: enumerate pins

If you have any question or need further information, send an email to service@thundercomm.com or raise your case on https://support.thundercomm.com/.

Thundercomm Confidential
2025-01-31 5:16:32 AM CST
Stefan_ucam.inforvision.com

Appendix 1. Notices

Thundercomm may have patents or pending patent programs covering subject matter described in this document. The furnishing of this document does not give you any license to these patents. You can send license inquiries to service@thundercomm.com.

THUNDERCOMM PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you.

Changes are made periodically to the information herein; these changes will be incorporated in new editions of the publication. To provide better service, Thundercomm reserves the right to improve and/or modify the products and software programs described in the manuals, and the content of the manual, at any time without additional notice.

The software interface and function and hardware configuration described in the manuals included with your development board or system on module might not match exactly the actual configuration of that you have purchased. For the configuration of the product, refer to the related contract (if any) or product packing list, or consult the distributor for the product sales. Thundercomm may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

The products described in this document are not intended for use in implantation or other life support applications where malfunction may result in injury or death to persons. The information contained in this document does not affect or change Thundercomm product specifications or warranties. Nothing in this document shall operate as an express or implied license or indemnity under the intellectual property rights of Thundercomm or third parties. All information contained in this document was obtained in specific environments and is presented as an illustration. The result obtained in other operating environments may vary.

The information of this document should not be as any invitation for offer or any advice to the visitors. Please consult the professional comments from the sales consultant prior to do any actions of investment or purchase.

Thundercomm may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Any references in this publication to non-Thundercomm Web sites are provided for convenience only and do not in any manner serve as an endorsement of those Web sites. The materials at those Web sites are not part of the materials for this Thundercomm product, and use of those Web sites is at your own risk. Thundercomm shall not be responsible for the content of the third party.

Any performance data contained herein was determined in a controlled environment. Therefore, the result obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

This document is copyrighted by Thundercomm and the property right of the date mentioned in this document, including but not limited trademarks, patents, copyrights, trade name etc. are not covered by any open-source license. Thundercomm may update this document at any time without notice.

Anyone doesn’t have the right to amend, reprint, republication, reproduce, transmit, distribute or any other way to use this document in business or public purpose without the prior written consent by Thundercomm.

E-mail messages sent to Thundercomm via the Internet are not guaranteed to be completely secure.Thundercomm shall not be liable for any loss incurred by the surfer when transmitting any information over the Internet or for any loss incurred by Thundercomm when sending any information over the Internet at your request.

Thundercomm has all rights under other relevant exemptions provided by laws and regulations, and Thundercomm's failure to claim or delay in claiming such rights shall not be deemed to be a waiver of such rights by Thundercomm.

Thundercomm reserves the right of final interpretation of this document.

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com

Appendix 2. Trademarks

Thundercomm, Thundercomm TurboX, TURBOX, Thundersoft turbox are trademarks of Thundercomm Corporation or its associate companies in China and/or other countries. Intel, Intel SpeedStep, Optane, and Thunderbolt are trademarks of Intel Corporation or its subsidiaries in the U.S. and/or other countries. Microsoft, Windows, Direct3D, BitLocker, and Cortana are trademarks of the Microsoft group of companies. Mini DisplayPort (mDP), DisplayPort, and VESA are trademarks of the Video Electronics Standards Association. The terms HDMI and HDMI High-Definition Multimedia Interface are trademarks or registered trademarks of HDMI Licensing LLC in the United States and other countries. Wi-Fi, Wi-Fi Alliance, WiGig, and Miracast are registered trademarks of Wi-Fi Alliance. USB-C is a registered trademark of USB Implementers Forum. All other trademarks are the property of their respective owners.

Thundercomm Confidential
2025-01-31 9:16:32 AM CST
Stefan_ucam.moorvision.com